High-Order Winslow smoothing/untangling

Andrew Shi

April 25, 2026

Contents

0 The 2016 formulation (brief recap) 2
0.1 Numerical Results/Tests . . . 2

1 W modification to achieve reference element shape 3
1.1 Target metric . . . 5
1.2 Modified split form . . . 5
1.3 Continuity of W and the $\tilde{\alpha}$ projection . . . 5
1.4 Numerical Results/Tests . . . 6
1.5 Summary . . . 8

2 Reference-mesh preprocessing via Tutte's embedding 8
2.1 Extracting the topology . . . 9
2.2 Tutte's barycentric embedding . . . 9
2.3 Shape optimisation of the straight-sided mesh . . . 10
2.4 Promotion to high-order reference . . . 11
2.5 Numerical examples . . . 12
2.5.1 Example 1: regular grid, `w2.m` topology . . . 12
2.5.2 Example 2: unstructured Delaunay on the unit square . . . 12
2.5.3 Example 3: smooth non-convex starfish . . . 14
2.5.4 Example 4: non-convex L-shape, two target shapes . . . 16
2.6 How to Present to Persson . . . 18

Codes live here.

0 The 2016 formulation (brief recap)

The Winslow equations in physical coordinates are

$$g^{ij}\,\partial_i\partial_j x_k = 0, \qquad k = 1, \ldots, n, \tag{1}$$

where g^{ij} are the contravariant metric components of the (nonlinear) mapping from the computational domain C to the physical domain D. The nonconservative form does not discretize cleanly in a CG framework, so the 2016 paper introduces auxiliary variables $\alpha_j = -\partial_i(g^{ij})$ and recasts the system in split form:

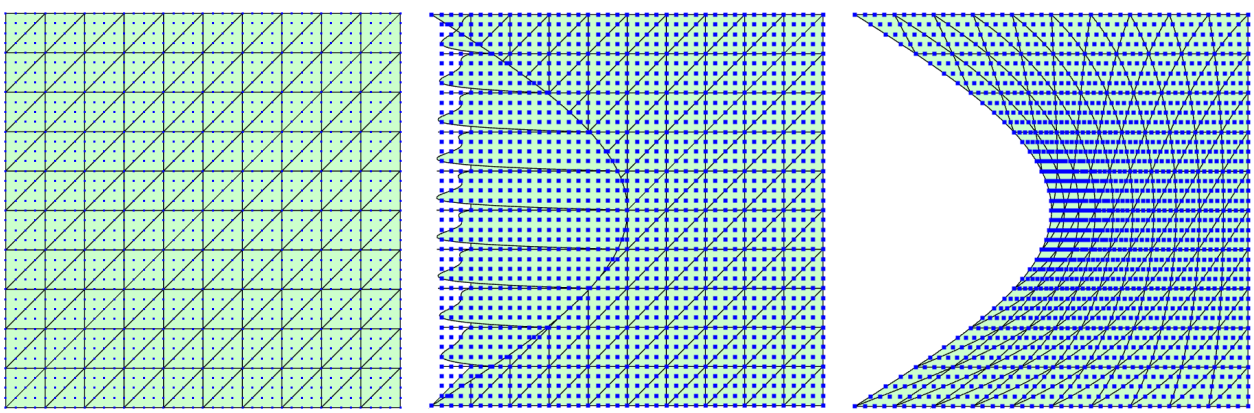
$$\partial_i(g^{ij}) + \alpha_j = 0, \tag{2}$$

$$\partial_i(g^{ij}\,\partial_j x_k) + \alpha_j\,\partial_j x_k = 0, \tag{3}$$

with Dirichlet data $\boldsymbol{x} = \boldsymbol{x}_{\text{bnd}}$ on ∂C. The point of this split form is to 1) mimic the what finite difference Winslow people are doing and 2) enable Picard iteration, which is the "magic" here.

0.1 Numerical Results/Tests

The code `w2.m` does the following in Figure 1.



(a) Reference mesh. (b) Initial configuration. (c) After Winslow

Figure 1: (a) Structured 11×11 reference mesh at $p = 4$. (b) Initial configuration: interior nodes unchanged, boundary on side 4 displaced by $0.5\sin(\pi y)$, producing heavy tangling. (c) After Winslow iterations, interior nodes have redistributed to absorb the boundary deformation; all elements valid.

To assess mesh quality we use the scaled Jacobian of Fortunato–Persson [1],

$$I \;=\; \frac{\min_{\xi\in\kappa} J(\xi)}{\max_{\xi\in\kappa} J(\xi)}, \qquad I \le 1, \tag{4}$$

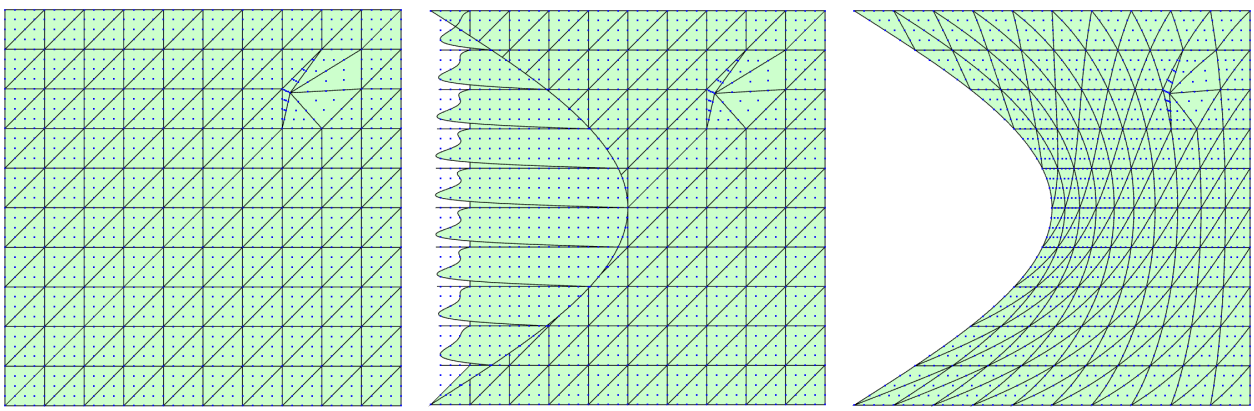
approximated at the Gauss quadrature points of each element κ. For a straight-sided simplex, J is constant across κ and $I = 1$ exactly; $I \le 0$ indicates a degenerate or inverted element.

To set up the main test case we will use throughout Section 1, we introduce `w2_skewed.m`, a slight variation of `w2.m` in which a single interior linear vertex of the otherwise uniform 11×11

reference is moved toward a neighbour, creating a small patch of sliver elements in the upper-right of the reference. Everything else—the boundary perturbation on side 4, the polynomial order, the smoother itself—is identical to `w2.m`. The scaled Jacobian I of (4) is blind to this regime: any simplex with straight edges has $I = 1$ regardless of aspect ratio, so it cannot "see" the sliver patch in the reference. To capture straight-sided shape distortion we also report the Liu–Joe inverse mean-ratio shape distortion metric [2, 3],

$$\eta \;=\; \frac{\|\boldsymbol{J}\|_F^2}{d\,|\det \boldsymbol{J}|^{2/d}}, \qquad \eta \in [1, \infty), \tag{5}$$

with $\eta = 1$ for a conformal (ideal) element and $\eta \to \infty$ as the element degenerates. The results are summarized in Figure 2 (meshes), Figure 3 (scaled Jacobian distribution), and Figure 4 (shape distortion distribution). Unmodified Winslow produces a valid physical mesh, but the sliver cluster visible in the reference persists as the worst element in the smoothed output. This is exactly the behaviour the W-modification of Section 1 is designed to address, and the single-sliver `w2_skewed` test is the setting in which we evaluate it.

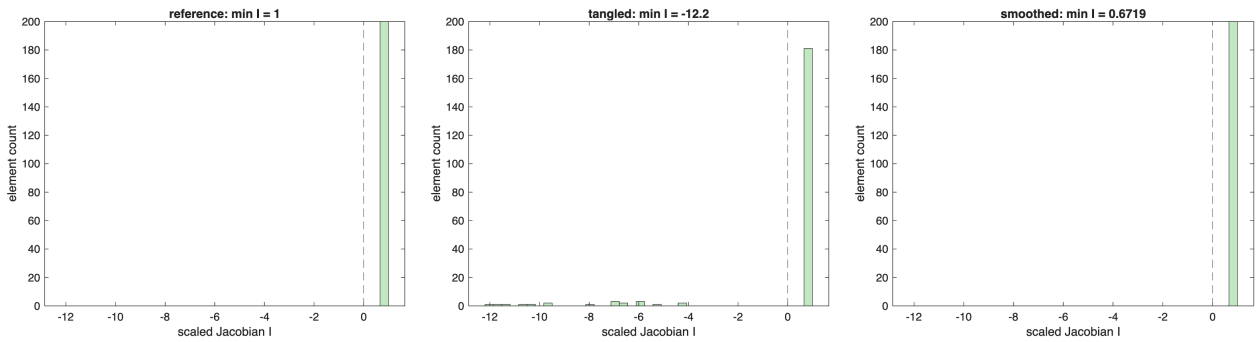


(a) Reference mesh. (b) Initial configuration. (c) After Winslow

Figure 2: Slight variation of `w2.m`: one interior vertex of the 11×11 reference at $p = 4$ is displaced toward a neighbour, creating a sliver cluster. (a) Skewed reference with $\max \eta = 4.63$. (b) Boundary on side 4 displaced by $0.5 \sin(\pi y)$, producing heavy tangling near the perturbed edge on top of the reference-induced sliver. (c) After unmodified Winslow: elements are valid, but the reference skew persists, with $\max \eta = 7.17$ on the sliver in the upper-right.

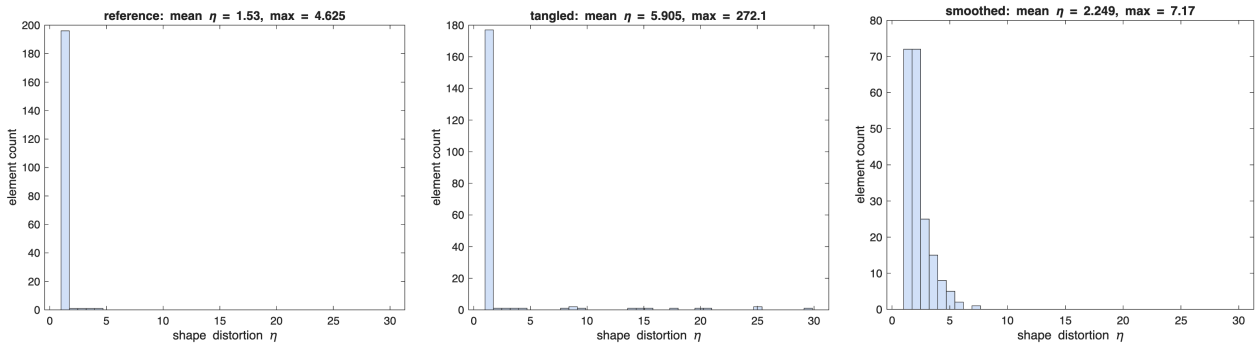
1 W modification to achieve reference element shape

The 2016 formulation produces a physical mesh whose element shapes are inherited from the reference mesh: the contravariant metric g^{ij} in the split-form system of Section 0 is built from the reference element geometry, so a skewed reference produces skewed physical elements. The W-modification below decouples the target shape from the reference element shape. **This is exactly the same formulation Persson gave me from his Gemini conversation (persson_gemini_winslow.pdf).**



(a) Reference (b) Tangled (c) Smoothed

Figure 3: Per-element scaled Jacobian $I = \min_\xi J(\xi)/\max_\xi J(\xi)$ for the `w2_skewed` test. (a) Reference: all elements straight-sided simplex, so $I = 1$ identically—the metric is blind to the sliver cluster present in the mesh. (b) Tangled: long negative tail, $\min I = -12.2$. (c) Smoothed: all $I > 0$, $\min I \approx 0.67$ on elements bordering the indentation.



(a) Reference (b) Tangled (c) Smoothed

Figure 4: Per-element shape distortion metric $\eta = \|J\|_F^2/(d\,|\det J|^{2/d})$ [2, 3] for the `w2_skewed` test. $\eta \in [1, \infty)$ with $\eta = 1$ at conformal (ideal) elements; unlike I, η detects straight-sided slivers. (a) Reference: $\max \eta = 4.63$, mean 1.53—the right tail is the sliver cluster around the moved vertex. (b) Tangled: $\max \eta = 272$. (c) Smoothed: $\max \eta = 7.17$, mean 2.25—the sliver cluster persists as the worst element in the smoothed output, motivating the W-modification in Section 1.

1.1 Target metric

For each reference element $K \in \mathcal{T}_h$, let $W_K \in \mathbb{R}^{n\times n}$ denote the Jacobian of an affine map from an ideal element (e.g. a unit equilateral simplex or unit square) to the reference element geometry. The total map from the ideal element to the physical element then has Jacobian $J_{\text{tot}} = JW_K^{-1}$, where $J = \partial x/\partial \xi$ is the reference-to-physical Jacobian.

The covariant and contravariant metrics of the total map are

$$\tilde{g}_{ij} = (J_{\text{tot}}^T J_{\text{tot}})_{ij} = (W_K^{-T} g W_K^{-1})_{ij}, \qquad \tilde{g}^{ij} = (W_K g^{-1} W_K^T)_{ij} = W_{K,ik}\, g^{kl}\, W_{K,jl}. \tag{6}$$

With $W_K = I$ for all K, $\tilde{g}^{ij} = g^{ij}$ and the 2016 formulation is recovered.

1.2 Modified split form

We replace g^{ij} by $\tilde{g}^{ij}$ in the split-form system of Section 0, introducing a modified auxiliary variable $\tilde{\alpha}_j = -\partial_i \tilde{g}^{ij}$:

$$\partial_i(\tilde{g}^{ij}) + \tilde{\alpha}_j = 0, \tag{7}$$

$$\partial_i(\tilde{g}^{ij}\, \partial_j x_k) + \tilde{\alpha}_j\, \partial_j x_k = 0, \tag{8}$$

with Dirichlet data $x = x_{\text{bnd}}$ on ∂C. The equivalence with the nonconservative form $\tilde{g}^{ij}\partial_i\partial_j x_k = 0$ follows from (7) by the same product-rule argument as in the unmodified case.

Heuristic nature of the extension. The classical Winslow system $g^{ij}\partial_i\partial_j x_k = 0$ is the reformulation, via change of variables, of $\Delta_x \xi_k = 0$—the Euler–Lagrange equation of the Dirichlet energy $\int_D |\nabla_x \xi|^2\, dx$ for the inverse map. The substitution $g \mapsto \tilde{g}$ in (7)–(8) is not, to the authors' knowledge, the Euler–Lagrange equation of any natural energy functional in x. In particular, for piecewise-constant W_K the map $\xi \mapsto W_K\xi$ is a per-element linear reparameterization of the reference coordinates, and harmonic maps are invariant under such reparameterizations, so the natural variational interpretation collapses to the unmodified Winslow system. We therefore regard (7)–(8) as a heuristic extension of the 2016 numerical scheme, parameterized so as to retain its Picard structure, rather than as the discretization of a derived PDE. Consequences: no Radó–Kneser–Choquet analog applies, and convergence of the resulting Picard iteration is assessed empirically.

1.3 Continuity of W and the $\tilde{\alpha}$ projection

The original Fortunato–Persson formulation already solves one projection problem: the auxiliary variable α_j is defined, through the weak form of $\partial_i g^{ij} + \alpha_j = 0$, as the L^2 projection of $-\partial_i g^{ij}$ onto the continuous CG space V_h^p. Operationally this is the linear solve $\alpha \leftarrow M\backslash b$, where M is the V_h^p mass matrix and b is assembled by quadrature from g^{ij} at Gauss points. The solve is what makes the scheme work on unstructured meshes. Because $x_h \in V_h^p$ is C^0 but not C^1, the gradient $\partial x/\partial\xi$ jumps across every internal edge, so g^{ij} is discontinuous there, and a strong derivative of g^{ij} would produce Dirac-style spikes supported on the interfaces. By omitting internal face fluxes in the weak form, the projection smears those spikes into a single continuous $\alpha \in V_h^p$, bypassing the DG machinery of numerical fluxes entirely. This is the computational magic of the 2016 paper.

In the modified system (7)–(8) this projection is reused without modification. $\tilde{\alpha}_j$ is the V_h^p projection of $-\partial_i \tilde{g}^{ij}$, computed with the *same* mass matrix M and the same backslash solve; only the right-hand side is altered to use $\tilde{g}^{ij} = W_K g^{ij} W_K^T$ instead of g^{ij}. No new projection infrastructure is introduced at this stage, and one can formally proceed without any further thought about the role of W_K.

The genuinely new object is W_K. It is the Jacobian of the affine map from the ideal equilateral triangle to the (straight-sided) reference triangle K—a map that sends the three corners of the ideal element to the three corners of K. Because the map is affine, its Jacobian W_K is a *single* 2×2 matrix, constant throughout the interior of K, at every polynomial order p; the polynomial order p controls how the *solution* x_h is represented, not the geometry of K, and the straight-sided reference keeps W_K element-constant regardless of p. The subscript K records the essential point that W_K is one matrix per element and jumps discontinuously across interfaces, since adjacent reference triangles are mapped from the ideal by different affine transformations.

At a glance it is tempting to conclude that these new jumps are absorbed for free by the $\tilde{\alpha}$-projection, just as the old jumps of g^{ij} were. The projection is indeed well-defined, but a subtler point intervenes. The jumps of g^{ij} at an interface are controlled by the jumps of $\partial x_h/\partial \xi$ there, and these shrink to zero as the Picard iteration converges, since the fixed point x_h is as smooth as the discrete space allows. FP's robustness relies on this self-limiting behaviour: the coefficient roughness that the iteration has to cope with decays alongside the iteration itself. The jumps of W_K are structurally different. They are set once and for all by the initial reference mesh, are external to the iterate, and *do not* decay with Picard. If the reference mesh contains neighbours of very different shape, the W_K jumps can be substantial, and we have observed empirically that the resulting persistent coefficient roughness destabilises the iteration.

The remedy suggested in the Persson/Gemini exchange is to elevate W to V_h^p *before* forming $\tilde{g}^{ij}$. At each CG node x_n, assemble a nodal target $W_h(x_n)$ as a weighted average of the W_K values from the surrounding elements,

$$W_h(\mathbf{x}_n) = \frac{\sum_{K \ni \mathbf{x}_n} |K|\, W_K}{\sum_{K \ni \mathbf{x}_n} |K|}, \tag{9}$$

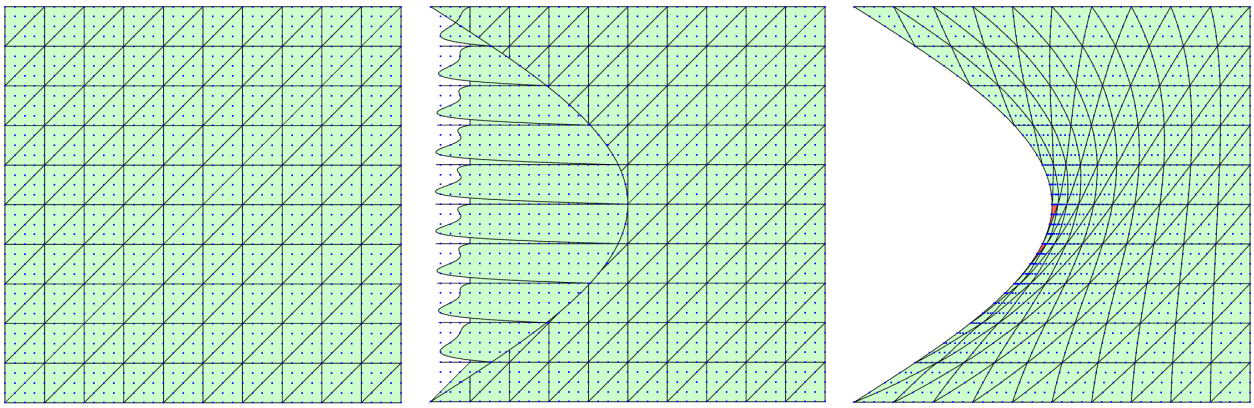
then interpolate via the standard shape functions. The resulting $W_h \in V_h^p$ is C^0, so $\tilde{g}^{ij} = W_h g^{ij} W_h^T$ inherits *only* the jumps already present in g^{ij}, which the $\tilde{\alpha}$-projection handles as before. The averaging itself is cheap: it is a lumped-mass L^2 projection onto V_h^p, equivalent to area-weighted nodal averaging. A full (non-lumped) L^2 projection would require an additional global mass-matrix solve but is not necessary, since the Winslow-like operator is relatively insensitive to moderate blur in the target metric.

1.4 Numerical Results/Tests

We first run the unskewed `w2.m` setup of Section 0.1 through the W-modified solver (`w2_W.m`). The reference mesh, boundary perturbation, and polynomial order are identical to `w2.m`; only g^{ij} is replaced by $\tilde{g}^{ij} = W_h\, g^{ij}\, W_h^T$, with W_h obtained by full (consistent) L^2 projection of the equilateral-target W_K onto V_h^p—the variant closest to the original 2016 formulation. The result is in Figure 5, to be compared directly to Figure 1.

The smoothed mesh has $\min J = -4.4 \times 10^{-4}$: two interior elements are slightly inverted (highlighted in red in Figure 5(c)), where `w2.m` produced $\min J = +3.1 \times 10^{-3}$ and all elements valid. A regression test (`w_regression.m`) verifies that $W_h \equiv I$ reproduces the unmodified solver bit-for-bit, so the inversion is from the formulation, not the implementation.

Running the same modification on the skewed reference (`w2_skewed_W.m`). Applying the same W-modified solver to the `w2_skewed` setup of Section 0.1—the sliver-cluster reference—produces qualitatively different behaviour: the Picard iteration does not converge. After an initial residual of ≈ 5, successive iterates oscillate in the range 0.4–5 for roughly thirty steps, then amplify



(a) Reference mesh. (b) Initial configuration. (c) After W-modified Winslow

Figure 5: `w2_W`: the `w2.m` setup run through the W-modified Picard solver with W_h constructed by consistent L^2 projection onto V_h^p. (a) and (b) are identical to their counterparts in Figure 1. In (c) the two red elements (centroids $(0.46, 0.37)$ and $(0.50, 0.47)$) have $\min J < 0$, so the W-modified output is inverted on a problem where the original 2016 formulation produced a valid mesh.

to residuals of 20–90, and finally hit the 100-iteration cap with no sign of settling. The unmodified solver, by contrast, converges cleanly on the same setup in 14 iterations (Figure 2). The divergence is not an implementation artefact; `w_regression.m` continues to pass.

This is the failure mode anticipated qualitatively in Section 1.3, where we noted that the jumps of W_K (or, more generally, the gradient of W_h) are *external* to the iterate and do not decay with Picard, unlike the jumps of g^{ij} which shrink as $\partial x_h/\partial\xi$ smooths. The product-rule expansion of $\tilde{\alpha}_j = -\partial_i \tilde{g}^{ij}$ makes this concrete. With $\tilde{g}^{ij} = W_h^{ik} g^{kl} W_h^{jl}$,

$$\tilde{\alpha}_j \;=\; -\,W_h^{ik} W_h^{jl}\,\partial_i g^{kl} \;-\; 2\,(\partial_i W_h)\cdot g \cdot W_h. \tag{10}$$

The first term is the original α conjugated by W_h. Like the unmodified α, it is driven by jumps in $\partial x/\partial\xi$, which shrink as x_h relaxes toward its fixed point—the iteration's self-limiting smoothing. The second term is *entirely new* and is independent of the current iterate: W_h is fixed once by the reference geometry, so $\partial_i W_h$ never decays with Picard. When the reference mesh is uniform, W_h is a constant field, $\partial_i W_h = 0$, and this new term vanishes—which is why `w2_W.m` converges and only exhibits a mild fixed-point inconsistency (the two inverted interior elements) rather than an iteration-level failure. When the reference contains a sliver patch, W_h varies sharply across a handful of elements and $\partial_i W_h$ is locally large; the new term contributes a persistent, iteration-independent forcing to $\tilde{\alpha}$ that neither the V_h^p projection nor the Picard loop can absorb.

Functionally, $\tilde{\alpha}_j$ is the advection coefficient in the x-equation $\partial_i(\tilde{g}^{ij}\partial_j x_k) + \tilde{\alpha}_j\,\partial_j x_k = 0$. Large, persistent $\tilde{\alpha}$ pushes the operator into the advection-dominated regime that standard continuous Galerkin treats badly without stabilisation (SUPG, upwinding, etc.), and in the Picard loop manifests as an increasingly-noisy update that eventually exceeds the iteration's natural contraction rate. The failure in `w2_skewed_W.m` is therefore not a quirk of our particular discretisation choice but a consequence of the W-modification itself once $\partial W_h \neq 0$.

This observation is important on its own, independent of any specific cure, because it explains why the scheme as literally written (following the Persson/Gemini exchange) cannot simply be "used". The iteration's self-limiting coefficient smoothing, which is what makes the 2016 formulation so robust, is broken by the W-modification in exactly the regime the modification is supposed to help. Before attempting to stabilise the iteration, it is worth understanding this.

1.5 Summary

The W-modification of Section 1, taken literally as the product-rule substitution $g^{ij} \mapsto W_h g^{ij} W_h^T$ in the 2016 split-form iteration, does not inherit the empirical robustness of the original formulation. Two reasons:

1. *Picard's self-limiting smoothing is broken.* The product-rule expansion of $\tilde{\alpha}_j = -\partial_i \tilde{g}^{ij}$ introduces a new term $-2\,(\partial_i W_h) \cdot g \cdot W_h$ that is independent of the current iterate. It persists as a smooth forcing in $\tilde{\alpha}$ that the L^2 projection cannot absorb and that Picard cannot relax, because it is not driven by $\partial x_h / \partial \xi$. Wherever W_h varies in space—which is exactly where the modification is supposed to do useful work—this term makes the x-equation advection-dominated and Picard non-contractive. Empirically, a single moved interior vertex in the `w2_skewed` reference is enough to send the iteration into oscillatory divergence. Smoothing W_h via lumped or consistent L^2 projection onto V_h^p reduces the magnitude of ∂W_h locally but does not eliminate it—projection trades interface spikes for interior gradients of comparable integrated magnitude. The only way to set $\partial W_h = 0$ is to take W_h globally constant, which defeats the purpose of the modification.

2. *There is no energy whose Euler–Lagrange equation is the modified split form.* A gradient of a scalar functional in x cannot have a non-zero iterate-independent piece, and the ∂W_h term is exactly that. This rules out any natural variational interpretation in x: there is no Lyapunov function to certify convergence, no minimum principle to guarantee non-inversion at the fixed point, and no standard machinery (line search, backtracking, Newton with globalisation) to stabilise the iteration. We are left with a heuristic fixed-point scheme without convergence theory.

The two observations are not independent: it is because the scheme is not a gradient that the ∂W_h term can appear at all. What makes the 2016 formulation's Picard so robust is precisely the self-consistency condition $\alpha_j = -\partial_i g^{ij}$, which makes α a computed consequence of the iterate; the W-modification breaks this by forcing $\tilde{\alpha}$ to absorb an external contribution. No amount of adjusting the W_h projection or the lumping recovers this structure. The modification is attractive on paper but the iteration machinery it was slotted into does not tolerate it.

2 Reference-mesh preprocessing via Tutte's embedding

Motivated by the negative findings of Section 1, we take a different approach. Rather than modifying the Fortunato–Persson iteration to correct for a poor reference mesh, we commit to using the 2016 scheme *as a black box* and instead construct a well-shaped reference mesh with the same topology.

The whole chain is:given a possibly tangled, poorly shaped high-order 2D mesh, we produce a straight-sided reference mesh with the *same connectivity* whose element shapes are as close to ideal as the topology permits, and then we feed that reference into the unmodified 2016 smoother with the original mesh's boundary as Dirichlet data.

The preprocessing chain has four steps, each taken up in a subsection below:

1. **Extract the topology** (Section 2.1). Discard the positions of the input mesh and keep only its combinatorial structure: the vertex list, the triangle list, and the identification of boundary vertices.

2. **Tutte's barycentric embedding** (Section 2.2). Map the boundary vertices to a convex polygon (e.g. the unit square), then solve Tutte's linear system for the interior vertex positions. The output is a straight-sided planar mesh with the input topology, guaranteed to have no inverted or degenerate elements.

3. **Mesh optimisation** (Section 2.3). *The key observation is that this step acts on a straight-sided, untangled mesh.* The Tutte output of Step 2 is guaranteed to have straight edges and positive element Jacobians, so standard mesh-quality optimisation—which is notoriously ill-posed on tangled or curved input—can be applied cleanly. We minimise a shape functional (e.g. the Liu–Joe shape distortion η (5)) while keeping the topology fixed and the vertex positions valid. This turns the "valid but possibly ugly" Tutte output into a "valid and well-shaped" straight-sided reference candidate.

4. **Promotion to high order** (Section 2.4). Apply `nodealloc` to allocate the V_h^p nodes on the straight edges of the optimised mesh, producing a valid high-order reference mesh that matches the input's polynomial degree.

The resulting reference mesh is then used as the input to `elliptic_smoothing.m` with the original (possibly tangled) mesh's boundary node positions as Dirichlet data. The 2016 scheme proceeds unmodified from that point.

2.1 Extracting the topology

Given the input high-order mesh we discard its geometric data—both the linear vertex positions and the high-order node positions—and retain only its combinatorial structure: the list of triangles (triples of linear-vertex indices) and the identification of which edges lie on the exterior boundary. This is enough to run Tutte's algorithm, which operates on the abstract planar graph of the mesh. Whether the input positions were tangled, inverted, badly shaped, or perfectly valid is irrelevant at this stage: the data Tutte sees is the same list of vertices and the same list of triangle connectivities.

Two structural assumptions on the input are required. First, the mesh must be topologically a disk—simply connected with a single exterior boundary. Second, its linear-vertex graph must be 3-connected. Both are satisfied by any standard triangulation of a simply connected planar domain with more than three vertices. Given these, the boundary forms a single cyclic loop of linear vertices, which we extract by walking the edges whose faces have no neighbour in the triangle-to-triangle connectivity. All remaining vertices are classified as interior.

2.2 Tutte's barycentric embedding

Tutte's theorem (1963): if the boundary vertices of a 3-connected planar graph are pinned to a convex polygon and each interior vertex is placed at the centroid of its graph neighbours, then the resulting straight-line drawing is a valid planar embedding—no triangle is inverted or degenerate. The theorem is striking in that it produces a globally valid mesh from purely local barycentric conditions, with no optimisation loop, line search, or sign-of-Jacobian tracking.

Given a chosen convex target polygon P (the computational domain we want our reference mesh to live in), the N boundary vertices extracted in Section 2.1 are distributed monotonically around ∂P. Each interior vertex $\mathbf{x}_i$ is then required to satisfy

$$\mathbf{x}_i \;=\; \frac{1}{\deg(i)} \sum_{j \sim i} \mathbf{x}_j, \qquad i \in \text{interior}, \tag{11}$$

where the sum is over the linear-vertex neighbours j of i and $\deg(i)$ is the valence. Collecting one such equation per interior vertex and moving the known boundary contributions to the right-hand side gives the sparse linear system

$$L_{\mathrm{ii}}\,\mathbf{x}_{\mathrm{int}} \;=\; -\,L_{\mathrm{ib}}\,\mathbf{x}_{\mathrm{bnd}}, \tag{12}$$

where $L = D - A$ is the graph Laplacian of the linear-vertex connectivity (D diagonal of valences, A the adjacency matrix), L_{ii} is its restriction to interior–interior entries, and L_{ib} its restriction to interior–boundary entries. The system is symmetric positive definite, and each of the two spatial components of $\mathbf{x}$ is solved independently with a single sparse direct call.

A few practical notes:

- Uniform arc length around ∂P is a reasonable default when no additional sizing or alignment information is available. Any monotone bijection from the input boundary loop onto ∂P works; replacing the uniform parameterisation with a size-aware one (e.g. matching the input's local boundary-edge lengths) is a mild extension.
- Tutte's output is uniquely determined by the topology and the boundary map. Distinct topologies produce distinct embeddings; distinct boundary parameterisations of the same topology produce distinct embeddings.
- The output is valid in the sense that every triangle has positive area, but it is not optimised for element shape. Vertices with high or irregular valence, or whose graph neighbourhood is highly asymmetric, typically produce short-edge or long-edge triangles. Improving those shapes is the job of Section 2.3.

2.3 Shape optimisation of the straight-sided mesh

The Tutte output of Section 2.2 is a valid straight-sided triangulation of the target polygon P: boundary vertices are pinned to ∂P, interior vertices sit at Laplacian positions, every element has positive area. We now relocate the interior vertices to improve element shapes. Two features of this step make it tractable: the input is untangled (every element has positive area) and straight-sided (every Jacobian J_K is a single 2×2 matrix per element). Both of these fail for the original input mesh—which is why this optimisation is deferred to after Tutte.

On each element K, the Liu–Joe shape distortion of Equation (5) reduces to

$$\eta_K \;=\; \frac{\|J_K\|_F^2}{d\,|\det J_K|^{2/d}}, \qquad \eta_K \geq 1, \tag{13}$$

where J_K is the Jacobian of the affine map from the ideal (equilateral) simplex to K. For the present linear problem J_K is an explicit, closed-form function of the three corner positions of K.

We aggregate these per-element values into a single scalar objective over the interior-vertex positions $\mathbf{x}$:

$$F(\mathbf{x}) \;=\; \sum_{K \in \mathcal{T}_h} \eta_K(\mathbf{x})^2. \tag{14}$$

Squaring, rather than taking the plain sum, has two practical effects. First, it penalises outlier elements quadratically, so a single near-degenerate element cannot hide inside a large overall sum of moderate values. Second, and more importantly, η_K diverges as $|\det J_K| \to 0$; the squared objective inherits this divergence and acts as an effective infinite barrier against element inversion.

Because the Tutte output in Section 2.2 gives a strictly positive Jacobian at every element, we start inside this barrier, and any descent direction that respects F stays inside it.

The problem is a standard unconstrained nonlinear optimisation over $\mathbb{R}^{2n_{\text{int}}}$ (two coordinates per interior vertex, boundary vertices held fixed). Gradients are analytic—J_K is affine in the three corner positions, so η_K is a rational function of the interior-vertex coordinates with explicit derivatives—and standard quasi-Newton solvers (L-BFGS, trust-region Newton, BFGS with Armijo line search) converge without special handling. The Tutte output is the initial guess.

This is the straight-sided specialisation of the curved-element mesh optimisation problem of Shi–Persson [3], whose Equation (4) generalises η_K to high-order elements by integrating a pointwise distortion over each curved element. In the present pipeline we do not need that generalisation because the mesh entering the optimisation step is straight-sided by construction.

Extension: sliding the target boundary. A natural extension is to drop the assumption that the boundary vertices are held fixed on ∂P. Instead, each boundary vertex is allowed to slide tangentially along the target boundary, keeping it on ∂P but freeing its arc-length position. For a circular target the boundary vertex is parameterised by an angle θ_i and its position becomes $(\cos\theta_i, \sin\theta_i)$; the objective gradient is extended through the chain rule. For a polygonal target each boundary vertex is parameterised by an arc-length along its currently-assigned edge, with corners pinned so that vertices cannot cross between edges. The added degrees of freedom let the optimiser redistribute boundary vertices to break the clumping that Tutte's uniform arc-length parameterisation can produce against the input topology.

When the physical domain is convex (or close to it), this extension is an unalloyed improvement. The reference-mesh quality rises across the board (on the examples below, mean η^2 drops by $\sim 45\%$ and the worst element falls by comparable amounts), and because the reference and physical boundaries have comparable geometry, the downstream Fortunato–Persson solver transports the improvement cleanly to the physical mesh. For non-convex physical domains with sharp concave features, the benefit is more nuanced: sliding tends to concentrate boundary vertices near a few regions that, under the proportional-arc map to the physical boundary, sit at the physical domain's concave neighbourhoods. Clustering near a physical concave corner can produce sliver elements whose effect on the final mesh outweighs the reference-quality gain. For the rest of this section we retain the pinned-boundary formulation as the primary pipeline; the sliding variant is optional and can be toggled per example.

2.4 Promotion to high-order reference

The output of Section 2.3 is a straight-sided linear triangulation of the target polygon P: fixed topology inherited from the input, boundary vertices pinned, interior vertices placed at a minimiser of the discrete shape objective. Nothing about the high-order structure has been touched yet.

Promoting to high order is trivial: run `nodealloc(msh, porder)` with the same polynomial order as the input mesh. The routine allocates high-order DG nodes by interpolating the shape functions on each straight-sided element from its three linear corner positions. Because the linear triangulation is straight-sided, there is no ambiguity: high-order nodes sit exactly where the element's barycentric interpolation places them, i.e., along straight edges and on a regular pattern in the interior of each element. The resulting high-order mesh inherits the element quality of the linear mesh from Section 2.3: every J_K of the reference element is still the same 2×2 affine Jacobian, and the Liu–Joe distortion η_K at each element is the same value it had at the end of the shape optimisation.

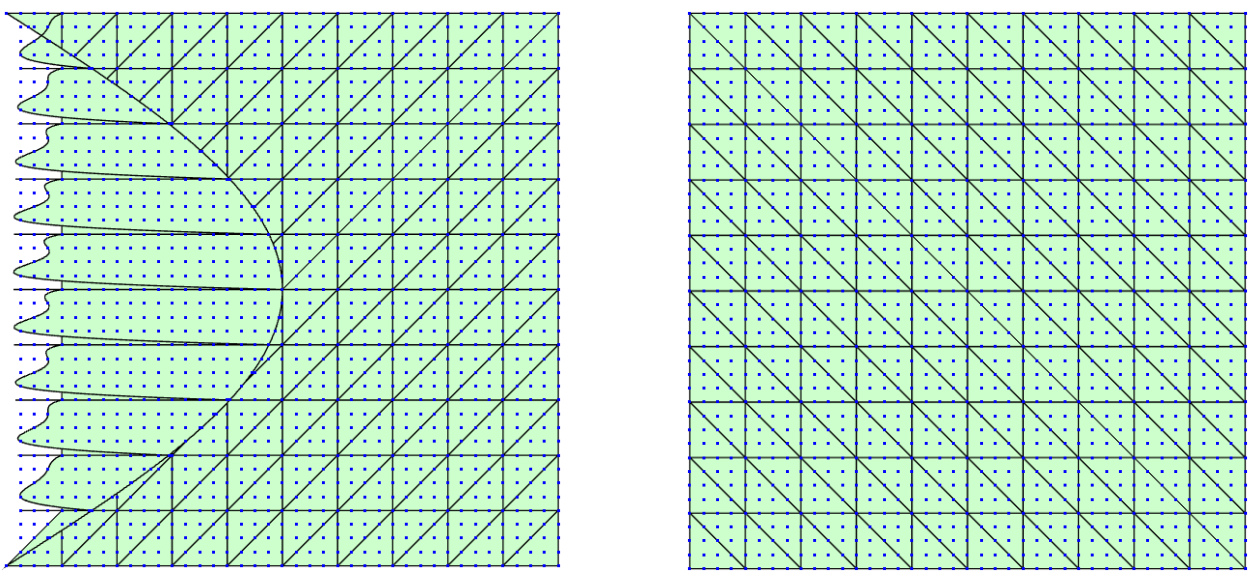
With this reference mesh in hand, we hand it to `elliptic_smoothing.m`, the original 2016 Fortunato–Persson solver, with the *original* input mesh's boundary node positions as Dirichlet data. No further modification is needed—the smoother runs in exactly the regime it was designed for: a well-shaped straight-sided reference with a prescribed (possibly curved) physical boundary.

2.5 Numerical examples

2.5.1 Example 1: regular grid, `w2.m` topology

We take the tangled input from `w2.m` (Figure 1, (b)) and feed it through the pipeline with no other information. In particular, the pipeline is *not* told that the input came from `mshsquare`; it sees a high-order mesh whose boundary is the unit square on three sides and the curve $x = 1 + 0.5\sin(\pi y)$ on side 4, with heavily tangled interior high-order nodes.

Figure 6(a) is the input. Tutte with target $= [0,1]^2$ produces (b)—the uniform 11×11 grid of `mshsquare(11, 11)`, recovered from topology alone, with every element at $\eta_K = 3/2$ (the Liu–Joe distortion of a right-isoceles triangle). Running BFGS on $F = \sum_K \eta_K^2$ with the equilateral simplex as the ideal element does not budge a node: the uniform grid is already a local minimum, $|\nabla F| = 4.6 \times 10^{-12}$ at iteration 0. Feeding (b) into `elliptic_smoothing.m` as the reference mesh, with (a)'s boundary as Dirichlet data, then reproduces the `w2.m` output of Figure 1(c).



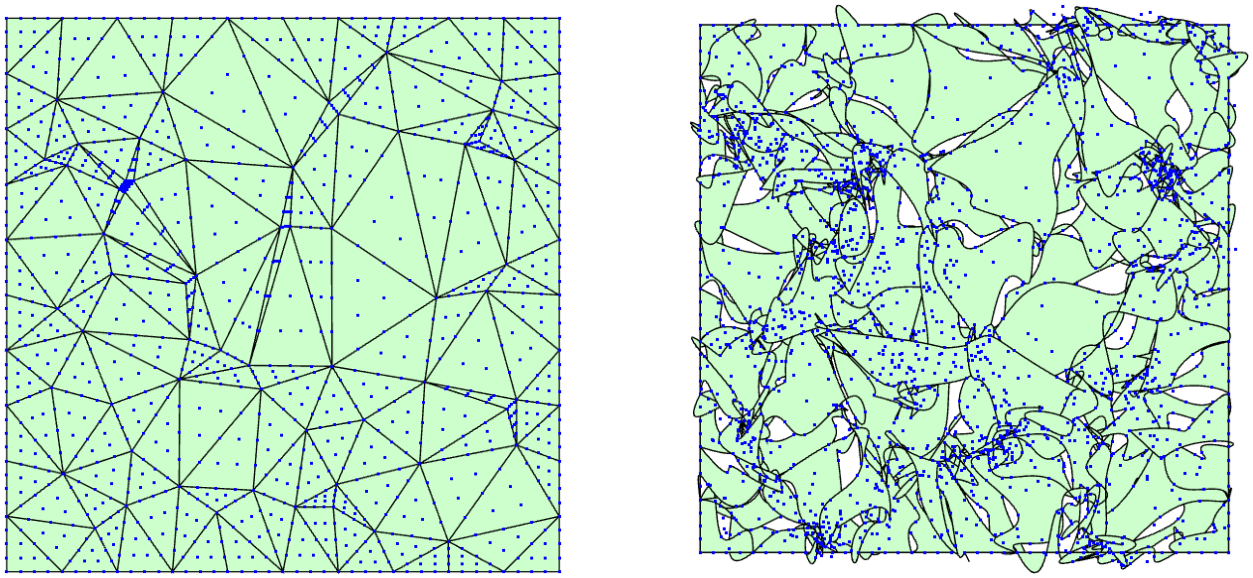
(a) Input: tangled mesh from `w2.m`.

(b) After Tutte with target $= [0,1]^2$.

Figure 6: `w2_tutte.m`: the pipeline applied to `w2.m`'s tangled input. (a) is identical to Figure 1(b), shown here for reference. (b) is the Tutte output.

2.5.2 Example 2: unstructured Delaunay on the unit square

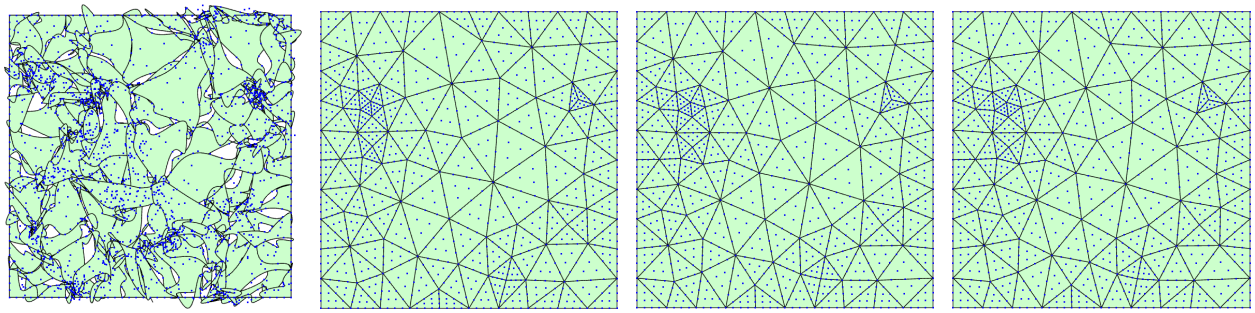
Incrementally harder: the same domain as Example 1 but with an unstructured topology, so the Tutte Laplacian becomes non-trivial and the shape optimiser has real work to do. The input is constructed in two steps shown in Figure 7: a clean Delaunay triangulation of the unit square with 40 boundary samples and 60 interior random points (a), then a mild tangling that perturbs each interior linear vertex by a uniform random displacement of amplitude 0.1 and each interior high-order DG node by amplitude 0.03 (b). Boundary linear and high-order nodes are left alone.



(a) Clean random Delaunay. (b) After mild interior tangling.

Figure 7: Construction of the Example 2 input: a clean random Delaunay triangulation of $[0, 1]^2$ (a) has its interior linear and high-order nodes mildly perturbed to produce the tangled input of the pipeline (b).

The pipeline proceeds through the four stages of Figure 8. Tutte (b) untangles the interior with a single sparse solve ($\max \eta = 2.64$, no inversions), placing each interior vertex at the centroid of its graph neighbours. Shape optimisation (c) redistributes the interior vertices with 137 BFGS iterations, lowering $\max \eta$ from 2.64 to 2.30 and F from 198.2 to 195.5; unlike Example 1, the optimiser has real work to do on an unstructured topology, though the two sliver fans at high-valence vertices (visible on the left and right of c) cannot be removed without retriangulation and remain as the worst elements. Handing the optimised reference to `elliptic_smoothing.m` with the tangled input's boundary as Dirichlet data (d) is trivial here—because the input's boundary was left unperturbed, it matches the reference boundary exactly, and Winslow converges in a single iteration reproducing (c).



(a) Tangled input. (b) Tutte. (c) Shape-opt. (d) Winslow.

Figure 8: Example 2 pipeline. (a) is the tangled input of Figure 7(b). (b) is the Tutte output. (c) is the output of shape optimisation on the Tutte output. (d) is the result of Fortunato–Persson Winslow smoothing with (c) as the reference mesh and (a)'s boundary as Dirichlet data; since the input boundary was left unperturbed, (d) is identical to (c).

2.5.3 Example 3: smooth non-convex starfish

Next: a smoothly curved, strongly non-convex boundary with no corners. We take a five-armed starfish,

$$r(\theta) \;=\; 1 + 0.40\,\cos(5\theta) + 0.03\,\sin(2\theta), \qquad (x, y) \;=\; r(\theta)\,(\cos\theta, \sin\theta),$$

with the principal $\cos(5\theta)$ term producing the five arms and the small $\sin(2\theta)$ modulation breaking exact 5-fold symmetry. The arm-to-bay radial ratio is $\sim 2.3:1$, so the arms are pronounced. We run the pipeline with the unit disk as the Tutte target.

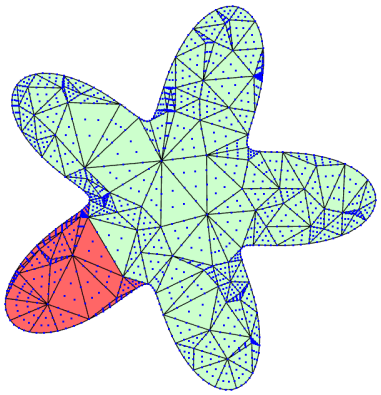
Figure 9 shows the pipeline in two parallel variants: the no-shape-opt path (column 2, Tutte directly into Winslow) and the with-shape-opt path (column 3, Tutte $\to$ shape-opt $\to$ Winslow). Column 1 is the physical starfish used by both paths. To make the geometric effect legible we highlight the largest connected clump of near-boundary triangles in the bottom-right sector of the Tutte output and trace the same triangle indices (preserved through Tutte and shape optimisation, since only vertex positions change) back to the physical mesh.

What is going wrong. Each arm of the physical starfish is a long, narrow protrusion occupying roughly 1/5 of the total area. Its perimeter share is comparable (arms and bays have similar arc length), so Tutte's proportional-arc-length mapping assigns the arm roughly $2\pi/5$ of the disk's angular extent. On the physical domain the arm's 27 triangles had reasonable shapes because the arm's 2D geometry—long radially, moderate tangentially—accommodated them. On the disk they must fit inside a thin angular wedge bounded by the arm's two boundary arcs on ∂P. The triangles connecting the arm's boundary to the disk's interior become long thin wedges emanating inward—the sliver fan of Figure 9(b).

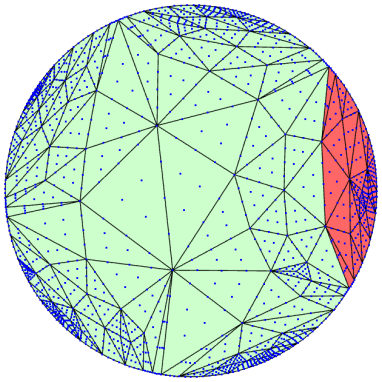
Why shape optimisation cannot fix this. The fixed-boundary shape optimisation of Section 2.3 can only move interior vertices. It cannot move the arm's boundary vertices from their Tutte-assigned positions on ∂P, and it cannot rewire the graph. Redistributing boundary density (which would require *boundary sliding*) and flipping the long diagonals of adjacent slivers (which would require *topological modification*) are both outside its reach. The residual near-boundary slivers in Figure 9(d) are structural to this step's feasible set, not a convergence failure.

The general principle. For a convex target P, a physical protrusion must fit into an angular wedge of P whose opening is set by the protrusion's perimeter share, not its area share. Unless those two shares match—they do not, for elongated protrusions—the interior triangles inside the protrusion get compressed tangentially into slivers on the reference. The Winslow step (e) undoes this compression when the physical boundary is smooth: the slivered wedge stretches back out along the arm and the elements recover moderate aspect ratios. This graceful transport is not guaranteed in general; in later examples we will encounter cases where it breaks down.

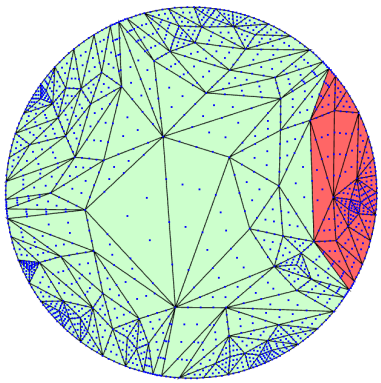
Aside: what shape optimisation actually optimises. Comparing the Winslow outputs with and without shape-opt reveals a subtlety about the objective. Shape optimisation minimises $\sum_K \eta_K^2$, and the Liu–Joe distortion η is *scale-invariant*: an equilateral triangle has $\eta = 1$ whether it is tiny or huge. Shape-opt therefore has no preference between "many small equilateral triangles" and "a few large equilateral triangles plus some boundary slivers"—it can trade the first for the second if the aggregate $\sum \eta^2$ is lower. By η-based metrics the shape-opt pipeline wins cleanly on our starfish run: mean $\eta^2 = 3.51$ versus 6.84 without shape-opt; maximum $\eta = 4.70$ versus 10.46. But visually, the no-shape-opt output can appear *more uniform* in element size, because its triangles are



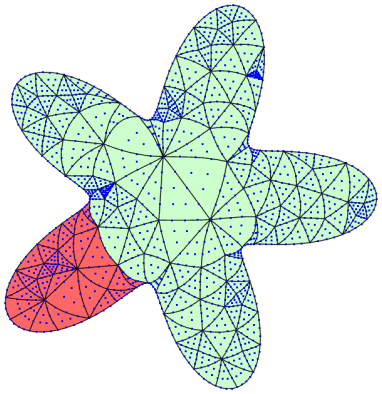
(a) Physical starfish. Red: an entire arm.



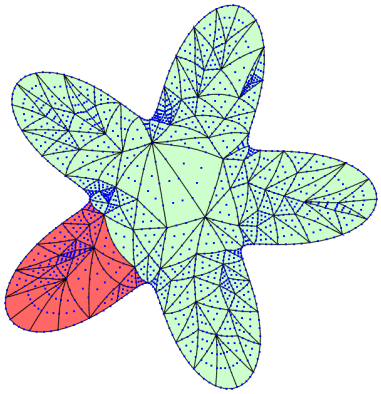
(b) Tutte (disk).



(d) After shape-opt.



(c) Winslow, direct from Tutte (no shape-opt).



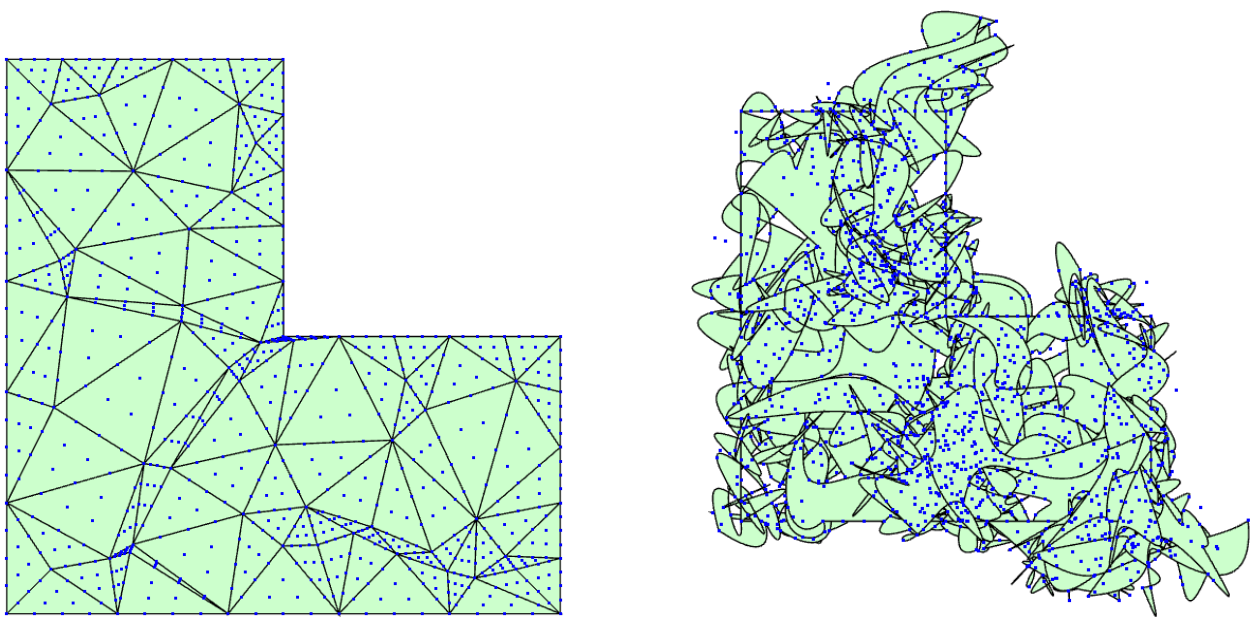
(e) Winslow, after shape-opt.

Figure 9: Starfish pipeline with unit-disk target. The 27 highlighted triangles fill one arm of the physical starfish (a). Column 2 is the *no-shape-opt path*: Tutte lays the arm's boundary vertices onto a thin angular wedge of the disk (b), producing an inward sliver fan, and Winslow transports that fan back to the physical arm (c) where it unwinds into a decent radial fan of moderate-aspect-ratio triangles. Column 3 is the *with-shape-opt path*: shape optimisation (d) compresses the wedge slightly but cannot reshape its sliver structure with the boundary pinned; Winslow then transports the polished wedge back to a similar radial fan (e). Shape-opt's contribution is a modest improvement of the worst element on the physical mesh (min $|J|$ $0.0011 \rightarrow 0.0026$), not a qualitative change; Winslow does most of the work.

smaller and more similarly sized across the domain. Visual size uniformity is not what $\sum \eta^2$ measures. If uniform element size is the goal—for a presentation figure or for a downstream solver that prefers homogeneous discretisation—a different objective is appropriate: $\sum \eta^2 + \lambda \operatorname{var}(\text{area}_K)$ is the simplest size-aware modification; a Gargallo-Peiró-style regularised distortion with an area target is a more principled one. Within the current pipeline, shape-opt remains correct as a local-shape objective but should not be read as a "visually best mesh" objective.

2.5.4 Example 4: non-convex L-shape, two target shapes

The next incremental step is a non-convex polygonal domain. We take the L-shape (unit square minus $[0.5, 1] \times [0.5, 1]$), which introduces a reflex corner at $(0.5, 0.5)$ and six boundary corners total. The input is constructed in Figure 10: a clean constrained Delaunay of the L with 30 boundary samples and 41 interior points (a), then a larger interior tangle (linear amplitude 0.25, high-order amplitude 0.08; Figure 10(b)).



(a) Clean L-shape constrained Delaunay.

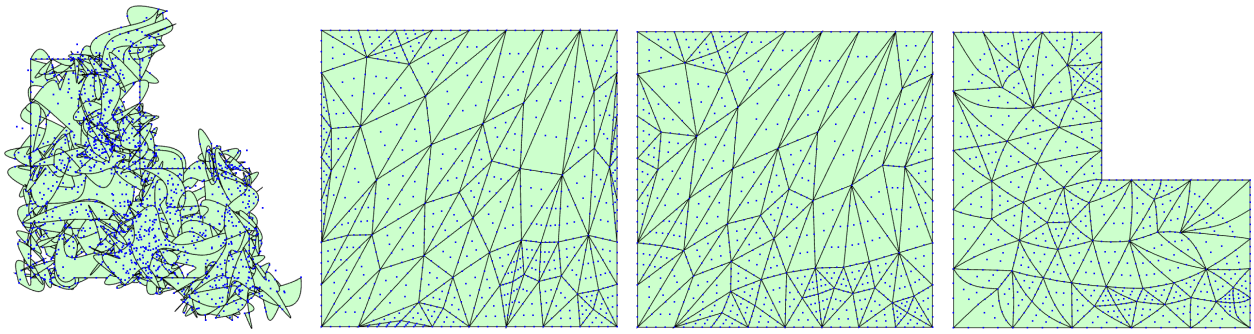
(b) After violent interior tangling.

Figure 10: Construction of the Example 4 input: a clean constrained Delaunay triangulation of the L-shape (a), with its interior linear and high-order nodes perturbed to produce the tangled input of the pipeline (b).

Because the L-shape is non-convex, the Tutte target P must be a convex polygon containing it topologically—some convex region we will use as the computational domain before handing off to Fortunato–Persson. We run the pipeline with two natural choices of P: the unit square and the unit disk.

Unit-square target. Figure 11 shows the four stages. Tutte with $P = [0, 1]^2$ produces a valid straight-sided reference (b, $\max \eta = 7.35$) on which shape optimisation does real work, pulling $\max \eta$ down to 4.15 over $O(10^2)$ BFGS iterations (c). Feeding this reference to `elliptic_smoothing.m` with the tangled input's L-shape boundary as Dirichlet data (d) produces a mesh of the physical L-shape, but with $\min I = -0.019$ (one element slightly inverted) and $\max \eta = 10.84$. The failure

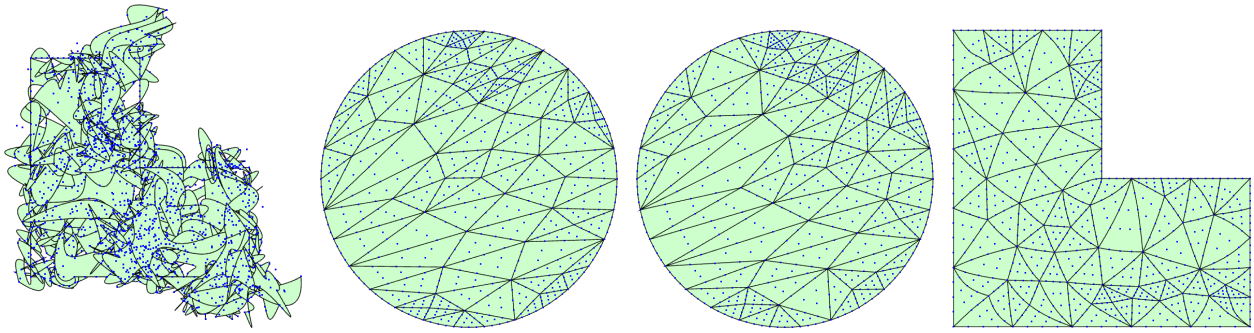
is geometric. The L has six corners but the square has four, so Tutte pins only four of the L's corners to target corners and the remaining two end up as regular boundary vertices on straight edges of the square. When Winslow maps the reference back to the L-shape, those two vertices must become sharp corners, and the neighbouring element absorbs the distortion.



(a) Tangled input. (b) Tutte (square). (c) Shape-opt. (d) Winslow.

Figure 11: L-shape pipeline with unit-square target. The square has four corners but the L-shape has six; Tutte pins only four of the L's corners to the target's. The Winslow step (d) produces a valid-looking L-shape mesh but with one slightly inverted element ($\min I = -0.019$) near a boundary vertex that had to become a sharp corner but was sitting on a straight square side in the reference.

Unit-disk target. Figure 12 repeats the pipeline with P the unit disk. Tutte distributes the L's boundary uniformly around the circle (b), shape optimisation settles to $\max \eta = 4.38$ over 113 BFGS iterations (c), and Winslow produces a clean L-shape mesh (d) with $\min I = 0.17$ and $\max \eta = 4.29$—all elements valid, and $2.5\times$ better in $\max \eta$ than the square-target output. The disk has no corners, so no boundary vertex is artificially assigned corner or non-corner status; the L's six corners are free to sit wherever they land on ∂P and Winslow reshapes them into sharp L corners without fighting the reference.



(a) Tangled input. (b) Tutte (circle). (c) Shape-opt. (d) Winslow.

Figure 12: L-shape pipeline with unit-disk target. No artificial corner alignment; the Winslow output (d) is valid ($\min I = 0.17$, $\max \eta = 4.29$) and more than $2.5\times$ better in $\max \eta$ than the square-target output of Figure 11.

2.6 How to Present to Persson

The setup

Fortunato–Persson 2016 assumes a valid straight-sided reference mesh is handed to you as input. Our motivation and question: I want to use Fortunato-Persson as is. As a black box. Can we construct the reference a valid and well shaped reference from the physical-domain boundary and the topology of the physical mesh, ignoring the actual physical positions of the vertex.

The pipeline

1. Input: a physical mesh (possibly tangled), or just a boundary polygon + linear-vertex topology from a Delaunay.
2. **Tutte's barycentric embedding**: fix boundary vertices on a convex target P (unit circle or unit square); interior vertices are the Laplacian solution. Straight-line embedding with no inverted elements, guaranteed by Tutte (1963).
3. **Shape optimisation** on the straight-sided Tutte output: minimise $\sum_K \eta_K^2$ over interior-vertex positions (boundary pinned), using BFGS with the equilateral triangle as the ideal.
4. **High-order promotion** via `nodealloc`.
5. **Winslow hand-off**: run the original Fortunato–Persson 2016 solver on the Step 1 input, using the result of Step 4 as the reference—our constructed well-shaped reference mesh.

Walking through the examples

- **Example 1 (Section 2.5.1): regular grid from `w2.m`.** Sanity check. The tangled input's underlying topology is a regular 11×11 grid of right-isosceles triangles; Tutte recovers exactly that grid from the topology alone, and shape-opt starts at a stationary point because the uniform grid is already a local minimum of $\sum_K \eta_K^2$. The Winslow hand-off then reproduces the canonical `w2.m` output.
- **Example 2 (Section 2.5.2): unstructured Delaunay on the unit square.** Here the physical domain and the Tutte target are both the unit square, so the Winslow step is trivial: given the constructed reference mesh, Winslow recovers it unchanged (identity map). The interesting part is the *Tutte solve and shape-opt*. Unlike Example 1 the topology is unstructured (random interior points, varying valences), so Tutte's Laplacian system is genuinely non-trivial and shape optimisation has real work to do to turn the Laplacian embedding into a well-shaped triangulation. This example validates the Tutte + shape-opt sub-pipeline as a reference-mesh generator in isolation from Winslow.
- **Example 3 (Section 2.5.3): smooth non-convex starfish.** The first case where every stage is doing non-trivial work. *Key difficulty*: each arm has a perimeter share comparable to the bays' but occupies roughly 1/5 of the area, so Tutte's proportional-arc mapping compresses each arm onto a thin angular wedge of the disk, producing an inward fan of slivers (Fig. 9b). Shape-opt cannot un-compress those wedges because the boundary is pinned; it only polishes interior positions. *How it still works*: Winslow's reference-to-physical map has bounded distortion on smooth boundaries, and the local Jacobian—radially-stretching, tangentially-gentle in an arm—composes with the slivered reference triangles to produce

moderate-aspect-ratio triangles on the physical arms (Fig. 9e). The pipeline outputs a clean physical mesh despite the unflattering intermediate geometry. *Cost of shape-opt*: the BFGS optimiser runs ~ 250 iterations with analytic gradients, and $\sum \eta^2$ drops by roughly half on the reference. By η-based metrics the shape-opt pipeline wins (mean η^2 3.51 vs 6.84; max η 4.70 vs 10.46). But the ablation in Fig. 9c shows the no-shape-opt Winslow output is visually comparable and can even look more uniform in element size—because η is scale-invariant, shape-opt has no preference for equally-sized elements, only for equally-*shaped* ones. See Section 2.5.3 for the detailed discussion and implications for size-aware objectives.

What works cleanly

- **Simply connected smooth non-convex domains (the starfish).** The pipeline produces dramatically cleaner physical meshes than the raw Delaunay input. The Winslow step transports the reference mesh back to the physical domain smoothly, and the per-arm sliver fans on the disk unwind into decent radial fans on the arms. The remaining artefact—a boundary-adjacent sliver ring on the disk, one sliver fan per physical arm—is structural to the convex-target choice and motivates the sliding / flipping extensions.

- **Simply connected with narrow pointy appendages** (hand, star, dolphin fluke). Tutte's theorem requires the linear-vertex graph to be 3-connected; pointy convex corners produce degree-2 boundary vertices that violate this. Fix: detect degree-2 boundary vertices, plant an interior anchor nudged inward from the boundary. Systematic detection, closed-form fix, works without manual tuning.

Extensions we explored

- **Boundary sliding in shape optimisation.** Let boundary vertices slide tangentially along ∂P during the optimisation (angles for circle, arc-lengths for polygon). Produces substantially better reference-mesh quality: on the L-shape, $\sum \eta^2$ drops by $\sim 45\%$ relative to the pinned-boundary version.

- **Proportional-arc Tutte + corner pinning + derive.** When the reference boundary slides, the Winslow hand-off loses reference–physical correspondence and Picard diverges. Fix: (i) map the input's boundary onto ∂P by proportional arc length so domain corners land at topologically-correct places; (ii) pin those corners during shape optimisation so they don't drift; (iii) at hand-off time, *derive* the physical-boundary Dirichlet data from the slid reference via proportional-arc mapping rather than reusing the input's sampling. The derive step is the component that makes Winslow converge; the others ensure the derived physical domain retains its correct geometry.

Where it breaks

- **Simply connected with sharp concave corners (L-shape).** The sliding extension improves reference quality, but the proportional-arc map transports boundary clustering from the reference onto the physical, and clustered vertices at a physical concave corner produce sliver outliers. Net effect on the L-shape: sliding improves the reference mesh ($\sum \eta^2$ down), but the final Winslow output is *worse* than the pinned-boundary pipeline at the worst element (max η jumps from 6.7 to 16.5). Pinned Winslow stays the primary pipeline for this class of domain.

- **Multiply connected domains.** Tutte requires a topological disk. A hole-slitting preprocess (`cut_to_disk`) is the classical remedy; we implemented and removed it. Currently out of scope.

Open directions

- **Edge flipping + smoothing alternation.** Lawson's flipping algorithm terminates at the Delaunay of the current vertex positions; alternating it with shape optimisation is known to drive $\sum \eta^2$ close to a local minimum that is "as good as it gets" for the fixed vertex set on the reference. Cheap to add, likely big win on the reference, unclear how much propagates to the physical.

- **Angle-gap barrier in sliding.** Soft penalty on clustering in the slide step so that the beneficial redistribution happens without the pathological clustering at concave-corner regions on the physical.

- **Physical-aware shape-opt objective.** Instead of minimising $\sum \eta^2$ on the reference, minimise a proxy for the post-Winslow mesh quality (e.g., $\sum \eta^2$ evaluated on the derived physical boundary). Addresses the reference-vs-physical mismatch directly rather than working around it.

Honest framing of the contribution

We are extending empirical FP-2016 to the case of reference-mesh-from-scratch. We inherit the same "works-in-practice-on-our-test-cases" standard as the 2016 paper; we do not prove convergence or optimality. The scope is simply connected 2D domains, with specific remedies for narrow appendages (anchoring) and curved targets (`msh.fd` hookup). Boundary sliding is a further extension that is an unalloyed improvement for convex or smooth physical domains, and a more nuanced tradeoff for sharp-cornered non-convex ones.

References

[1] Meire Fortunato and Per-Olof Persson. High-order unstructured curved mesh generation using the Winslow equations. *Journal of Computational Physics*, 307:1–14, 2016.

[2] Anwei Liu and Barry Joe. Relationship between tetrahedron shape measures. *BIT Numerical Mathematics*, 34(2):268–287, 1994.

[3] A. Shi and P.-O. Persson. Local element operations for curved simplex meshes. *International Journal for Numerical Methods in Engineering*, 2023. Author manuscript.